Knowledge-Based Systems 316 (2025) 113341

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Transformer-based statement level vulnerability detection by cross-modal fine-grained features capture☆

Wenxin Tao, Xiaohong Su*, Yekun Ke, Yi Han, Yu Zheng, Hongwei Wei

Faculty of Computing, Harbin Institute of Technology, Harbin, China

ARTICLE INFO

Keywords:
Software security
Multimodal deep learning
Vulnerability detection
Fine-grained vulnerability localization
Code slice purification

ABSTRACT

Software vulnerability detection is crucial for computer systems. The use of fusion information from the source and assembly code can improve the performance of deep learning-based vulnerability detection; however, the bimodal fine-grained alignment information has not been fully utilized in existing methods. Therefore, we propose a cross-modal fine-grained feature capture method based on Transformers for statement-level vulnerability detection. First, we apply a slice generation method based on cross-slicing of bimodal code to obtain bimodal slices. Second, we propose a bimodal-based slice purification method that can effectively shorten the length of source code slices, thereby reducing the impact of vulnerability-unrelated statements on the detection performance. In addition, we replace the information in assembly code that is not conducive to semantic understanding with more easily understandable information, which allows the model to accurately capture the vulnerability features in assembly code. Finally, the fine-grained aligned and purified bimodal code slices are input into the bimodal Transformer model, which can not only capture the vulnerability features within the statements of the bimodal code but also capture the long-term dependencies between statements through context. Compared with existing SOTA methods, the proposed method achieved an average improvement of 6.5% in IOU on the real-world project dataset.

1. Introduction

In recent years, deep learning has significantly improved vulnerability detection tasks. Common deep learning-based vulnerability detection methods include token sequence-based methods [1,2], graph neural network (GNN)-based methods [3,4], and bimodal-based methods [5]. However, these methods have some limitations. First, methods such as those of Peng et al. [6] only output statement lines most likely to be vulnerabilities. For code gadgets containing multiple lines of vulnerability statements, it is impossible to locate the structure that triggers the vulnerability, which makes this method lack practicality. Second, the source code can more intuitively represent control flow. For example, *if* and *while* statements directly indicate the purpose of the control flow, it is easier to detect logical vulnerabilities. Assembly code operates directly on CPU registers, providing clear paths for data transfer between them. It can help identify potential vulnerabilities that are difficult to detect in source code, such as memory leaks and stack overflows. These two code modalities can complement each other at different levels; thus, they work together to discover more types of potential vulnerabilities. In our previous work [5], the use of cross-modal feature enhancement and fusion of source and assembly code achieved good results in the task of code gadget vulnerability detection. However, it did extend to achieving statement-level vulnerability detection results. The input assembly code includes redundant information (e.g., register names) that is not conducive to semantic understanding, whereas the input source code, although subject to the process slicing criterion, still excludes vulnerability-irrelevant statements to enhance the performance of feature capture. Third, some methods have achieved good results using the Transformer architecture. For example, Fu et al. [7] proposed a Transformer-based approach for statement-level vulnerability detection, and Him et al. [8] proposed a statement-level vulnerability detection method based on Transformers combined with GNNs. However, none of them used fine-grained specialized evaluation metrics, such as Intersection over Union (IOU), which can accurately evaluate the performance of fine-grained vulnerability localization methods. In the vulnerability localization task, IOU metrics can be used to evaluate the degree of overlap between the detected and actual vulnerability locations and thus assess the

☆ This work was supported by the National Natural Science Foundation of China Grant nos.62272132.

* Corresponding author.
E-mail addresses: 21B903092@stu.hit.edu.cn (W. Tao), sxh@hit.edu.cn (X. Su), 22S103264@stu.hit.edu.cn (Y. Ke), yihan@ir.hit.edu.cn (Y. Han), 24S136058@stu.hit.edu.cn (Y. Zheng), 20B903043@stu.hit.edu.cn (H. Wei).

https://doi.org/10.1016/j.knosys.2025.113341
Received 20 December 2024; Received in revised form 13 February 2025; Accepted 10 March 2025
Available online 21 March 2025

accuracy of vulnerability localization. To address the above problems, we propose a purification method based on bimodal slicing of the source and assembly code, and we design a bimodal Transformer model to capture fine-grained vulnerability features to locate multiple vulnerable statement lines that jointly trigger vulnerabilities. The primary contributions of this paper are as follows:

1. **A new framework:** A vulnerability localization method based on fine-grained feature capture in the source and assembly code is proposed. Sufficient utilization of bimodal fine-grained alignment information and a new vulnerability detection framework based on the Transformer encoder–decoder architecture of bimodal features better captures vulnerability features, making it applicable to the task of detecting multiline vulnerabilities.
2. **Bimodal based slice purification method:** This method is used according to the features of the source and assembly code. We used cross-slicing of the bimodal code to obtain the slices of the source and assembly code. For the source code, slice lengths are shortened using the purification method, which helps to effectively exclude nonvulnerable code information. For the assembly code, information that hinders semantic understanding is replaced with information that enhances it. This helps the model to rapidly capture vulnerability features and improve localization performance.
3. **Extensive experiments and bimodal datasets:** The proposed method was not only tested on publicly available datasets (i.e., FFmpeg and OpenSSL) but also supplemented with new datasets from PostgreSQL version 9.2.4 and Apache Subversion version 1.8.3b. The expanded Twin Peaks Real Project dataset contains a more comprehensive set of vulnerability types. We validated the effectiveness of the proposed approach through extensive experiments and compared it to SOTA methods using specialized evaluation metrics at the code gadget-level and statement-level detection. The dataset and source code we used can be downloaded from GitHub (https://github.com/alg4ahvnt/postg-svn).

2. Background and motivation

2.1. Motivation example

An example exploiting code from the SARD dataset is shown in Fig. 1. The assembly code in Fig. 1(b) corresponds to the source code mapping line numbers shown in Fig. 1(a), where the comment of each assembly instruction contains information about the registers in the assembly instruction corresponding to the variables in the source code statement. Because of limited space, Fig. 1(b) shows only the sequence of assembly instructions corresponding to the statements in the source code vulnerability function *test()*. The vulnerability example is a complex piece of vulnerable code triggered by multiple vulnerable statements, containing an integer overflow vulnerability and a buffer overflow vulnerability. In Lines 9 and 10, i.e., a loop in the source code, the value of i at the end of the for-loop in Line9 is n because there is no check to see whether i exceeds the boundary of *buf*. In Line 12, if the value of n is greater than the actual allocated size of *buf*, buffer overflow occurs, resulting in a subscript overrun when *buf[n]* is accessed. Changing *buf[i]* in (Line12) to *buf[i-1]* fixes the vulnerability in this line. The intent of the check in Line 20 is to prevent integer overflow, and in practice, the primary method to fix the bug is to add an *if* condition [9]. However, the *if* statement added in Line 20 is invalid, because when n is negative, the *if* condition in Line 20 is still true, and then *test()* is called and executed. Because the parameter n is an unsigned integer, passing a negative number to n will be interpreted as a very large unsigned integer, which in turn will result in a possible failure of the allocation, as there may not be enough space in Line6 for a dynamic memory allocation.

From an assembly code perspective, Line 6's corresponding assembly instruction, including the call *malloc@PLT* instruction, is used to request memory allocation. If the allocated memory size is not calculated correctly, fewer memory resources may be allocated than expected. In the instruction *mov eax, DWORD PTR -20[rbp]*, we use the fusion information method to add comments to understand it as the instruction that writes data i to the buffer. If proper boundary checking is not performed, the above instructions may cause buffer overflow. Line 20's corresponding assembly instructions (i.e., the *jmp .L4* and *cmp* instructions) are used to control loops and conditional execution. If the loop condition is not properly set buffer access may exceed its allocated boundaries. The source and assembly code exhibit different vulnerability features, and each can be analyzed to determine the existence of a vulnerability.

From the above example, the following observed findings can be drawn:

1. The code gadget shown in Fig. 1 contains multiple lines of vulnerability constituting a complex vulnerability structure. The complex structure increases not only the difficulty of vulnerability detection but also the difficulty of accurately locating the vulnerability statements. However, both the source code and the assembly code can identify vulnerabilities using different features. Using fine-grained alignment information, assembly code can help locate the vulnerability statement, just like the source code.
2. Compiled source code statements without the corresponding assembly instructions (i.e., Lines 1–4 and 8) did not operate on the compile preprocessor; thus, these statements for vulnerability detection can be regarded as noise.
3. The readability and comprehensibility of assembly instructions are not as good as those of the source code because the operands in assembly instructions are represented by temporary variables or registers. If these operands are replaced with source code variables, the semantics of the assembly code become easier for the model to understand, making the vulnerability features extracted from the assembly code a useful supplement to those extracted from the source code.

2.2. Motivation for fine-grained cross-modal feature capture

In our previous work [5], we proposed a new vulnerability detection method based on multimodal deep learning, which can effectively extract vulnerability-related cross-modal features by compiling the source code to generate assembly code aligned with its fine-grained alignment. The assembly code obtained using traditional methods is a set of consecutive instructions, and it is not known which instructions correspond to which source code lines, making it difficult to effectively utilize the characteristics of assembly code in a fine-grained vulnerability localization task. Moreover, the variable/parameter information is deleted, and register information is introduced in the compilation preprocessing stage, which creates a lexical gap between the source code and the assembly code. To address this issue, we retain the variable/parameter information and register information during code preprocessing and align the instruction sequence of the assembly code with the statement lines of the source code, thereby allowing the proposed method to combine and fully utilize the bimodal features captured from both source code and assembly code in the fine-grained vulnerability localization task. The fine-grained alignment information is primarily used for code slice generation; however, it is not used in slice-level vulnerability detection. The fine-grained vulnerability localization task requires locating the specific statement that triggered the vulnerability; thus, fine-grained alignment information is very useful for the fine-grained vulnerability localization task. This is because the fine-grained alignment information, which contains statement and instruction sequence alignment information, as well as register and variable alignment information, helps capture the interconnections of fine-grained vulnerability features between the two modalities, leading to more accurate and efficient localization of vulnerable statements.

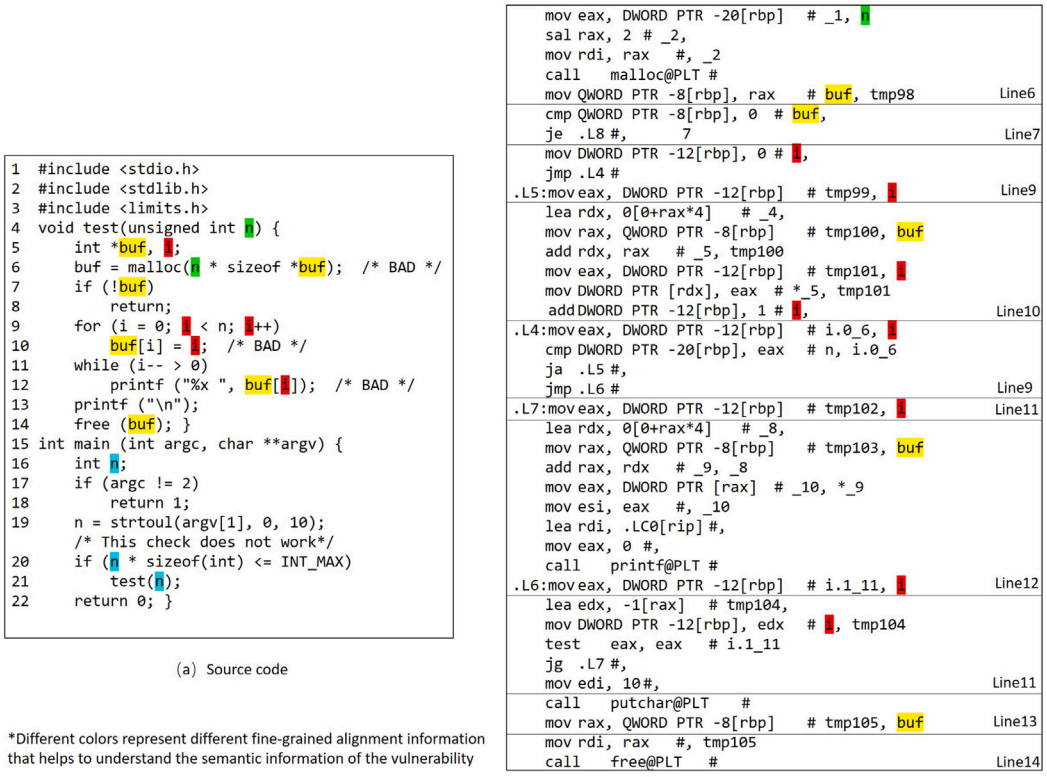

Fig. 1. Example of fine-grained alignment vulnerabilities between source code and assembly code from the SARD dataset.

2.3. Motivation for the Transformer architecture

The excellent performance of the Transformer architecture was demonstrated in NLP tasks, including machine translation and text summarization tasks. The key feature of the Transformer architecture is the self-attention mechanism, which allows us to capture long-term dependencies in the input sequence and understand complex sentence structures and semantics. It is important to identify multiline vulnerability statements distributed at different locations in code gadgets. This information has not been fully utilized in the current vulnerability detection methods, and no studies have applied the Transformer architecture to fine-grained vulnerability detection based on the source and assembly code. Therefore, we designed a Transformer architecture suitable for bimodal code inputs.

2.4. Motivation for bimodal-based slice purification

The syntax of the assembly code is completely different from that of the source code. To make the semantics of the assembly code easier for the model to understand, we propose a bimodal-based slice purification method to optimize the assembly code, integrating the semantic information from the source code without changing its structure. Moreover, some pretrained deep learning models that can process both natural language and source code (e.g., CodeBERT [4]) support several programming languages; however, they do not natively support assembly languages. We use the slicing purification method to make the pretrained model friendlier to support assembly languages.

First, the assembly code slices must be purified. The assembly code consists of a series of assembly instructions, each consisting an operator and an operand. The operand information defined in the nonsource code is introduced in the compilation preprocessing stage. This information creates a lexical gap between the assembly code and the source code, which makes it difficult for the deep learning model to understand the semantics of the assembly code. Making the model learn the operand information, such as registers, reduces learning efficiency and effectiveness. In this study, we propose a purification method for assembly code, where operand information is replaced by comments containing variable information to help bridge the lexical gap between the source code and the assembly code. The purified assembly instructions will be represented as phrases to help the model better understand the deep semantics of the assembly code, strengthening the correlation between the assembly code and the source code.

Second, the source code slices must also be purified. This is because we found that after the source code is preprocessed and optimized by the compiler, some statement information that does not produce the corresponding assembly code are removed. For example, compilation preprocessing directives (e.g., *#include*) are handled at the preprocessing stage of compilation, and assigned variables are retained during compilation preprocessing and replaced with pseudo-directives, such as *BYTE WORD*. When the operands are called for the first time, the assembly code reflects the execution process of a program and usually contains the performed operations. The source code statements without the corresponding assembly code are unlikely to result in vulnerabilities; thus, statements that do not involve actual execution of operations are noise to the vulnerability detection model, and the performance of this model can be improved by removing these statements. Therefore, we propose a source code slice purification method based on the characteristics of the assembly code execution process.

3. Overview

In this paper, we propose a fine-grained vulnerability detection method for multimodal code based on the Transformer encoder–decoder architecture. Fig. 2 shows the framework of the proposed method, which comprises three parts: data preprocessing, model training, and model testing.

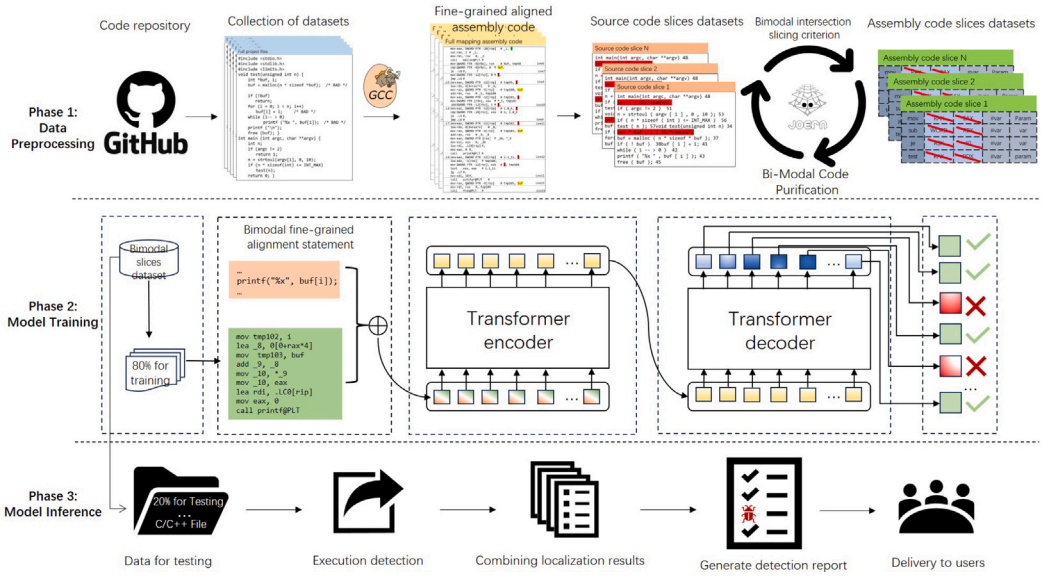

Fig. 2. Overview of the Transformer-based statement level vulnerability detection by cross-modal fine-grained features capture.

The data preprocessing phase, as shown in the first phase of Fig. 2, prepares the dataset for training the vulnerability detection model. First, the complete project dataset was collected from code repositories, namely, GitHub [10] and National Vulnerability Database (NVD) [11], and vulnerability statements were labeled. Then, we compiled the source code to generate its assembly code and obtain the assembly instruction sequences that were aligned with the source code statements. We preprocessed the bimodal code via bimodal cross-slicing and bimodal slicing purification to obtain a purified bimodal code slices dataset. The dataset was divided into training and testing sets with an 8:2 ratio.

The model training phase, as shown in the second phase of Fig. 2, trains fine-grained vulnerability detection models that can accurately predict the location of vulnerability statements. The proposed fine-grained vulnerability detection model based on the Transformer encoder–decoder architecture was trained on a multimodal training dataset.

The testing phase, as shown in the third phase of Fig. 2, obtains an accurate detection report. The model was tested on a testing dataset created during the first phase. Then, the code to be detected was generated into bimodal code slices, which were then input into the trained model for vulnerability statement localization. Finally, the collection of vulnerability statements in the detected sliced code gadget was labeled into the original source code file, and a detection report was generated and delivered to the user for confirmation.

4. Methodology

4.1. Data preparation and dataset construction

In our previous study, the bimodal vulnerability code dataset collected and created using command-line compilation contained only coarse-grained vulnerability label information, which could be used only for coarse-grained vulnerability detection. To make it suitable for fine-grained vulnerability detection, we added labeling information for vulnerability statements to the dataset. The specific steps are as follows:

First, we collect the test suite from the NVD website and extract the test cases separately. The test case file is a single c/cpp file. We locate the corresponding complete open-source project in GitHub, replace the test case c/cpp file with a corresponding c/cpp file in the complete project, and prepare it for compilation. Second, the GCC compiler assigns appropriate registers and register locations to variables/parameters during the preprocessing phase of the compilation, and we retained the above information. Third, we use the GCC compiler's single-step debugging technique to label the line number of the source code statement corresponding to the assembly code. Finally, we merge the information generated in the first two steps to generate assembly code containing the bimodal fusion information. The bimodal fusion of information refers to establishing a many-to-one statement alignment relationship between the instruction sequences in the assembly code and the statements in the source code, as well as a one-to-one mapping between the registers in the assembly code and the variables in the source code. We use uniform compilation parameters in the aforementioned stages to minimize the external interference of the experiment.

4.2. Slice purification based on cross-slicing of the bimodal code

4.2.1. Purification of source code slices after bimodal cross-slicing

In order to further reduce the length of source code slices and minimize vulnerability-unrelated source code statements in code slices, we propose a source code slices purification method based on the source code slices criterion. The method takes advantages of the execution characteristics of the assembly code, making the source code slices more concerned about the vulnerability-related features. The specific steps are as follows:To further reduce the length of source code slices and minimize vulnerability-unrelated source code statements in code slices, we propose a source code slice purification method based on the source code slice criterion. The proposed method exploits the execution characteristics of the assembly code, thereby increasing the level of concern regarding source code slices related to vulnerability-related features. The specific steps are as follows: First, the source code is parsed using a static analysis tool to generate an abstract syntax tree (AST) and program dependency graph (PDG). Second, the vulnerability candidate key points are extracted from the AST of the source code and are treated as the source code slicing criterion. A collection of slicing statements of the source code is obtained by bi-directionally traversing the PDG, generating unpurified source code slices based on the source code slicing criterion. Third, based on the bimodal fusion information obtained (Section 4.1), specifically the mapping relationship between the source code and the assembly code, the complete sequence of assembly instructions corresponding to each statement in the source code slices is mapped to generate unpurified assembly code slices

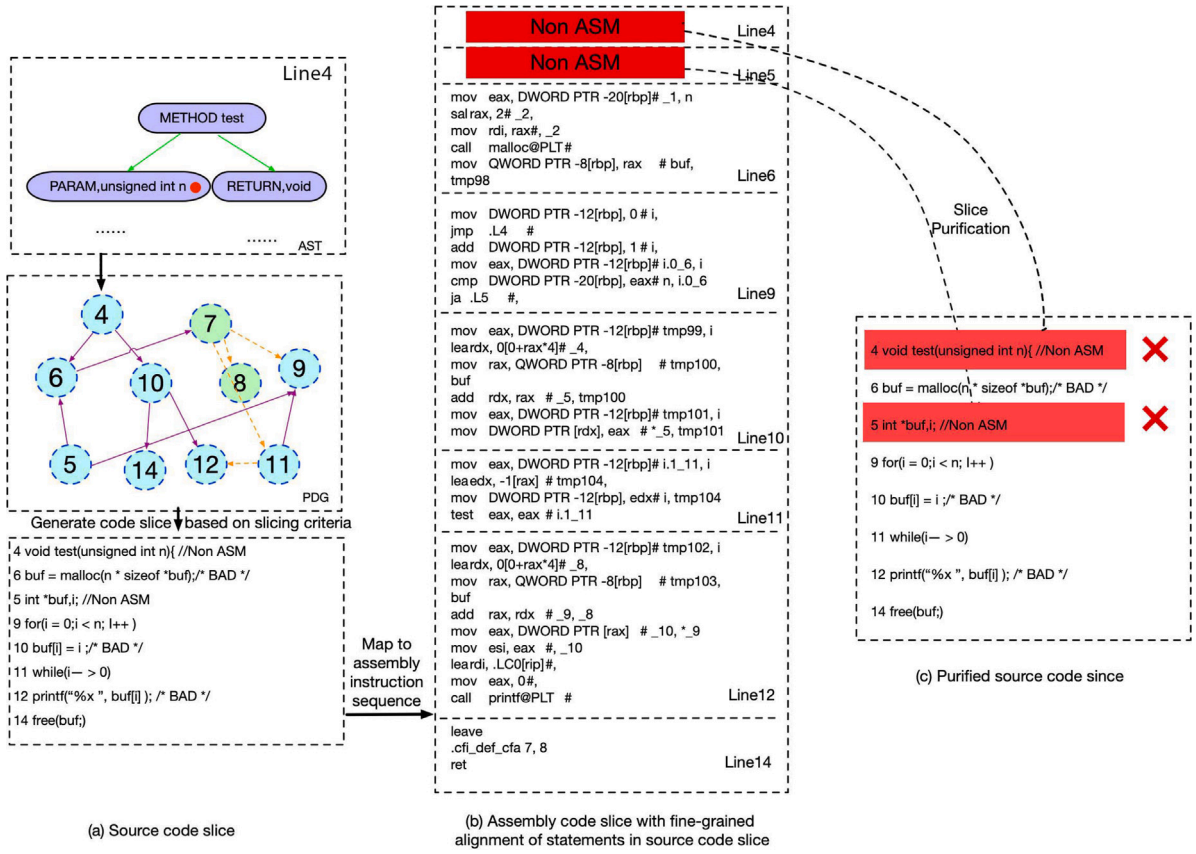

Fig. 3. Overview of the Transformer-based statement level vulnerability detection by cross-modal fine-grained features capture.

extracted according to the source code slicing criterion. Finally, the source code slice is purified by removing the source code statements without the corresponding assembly instruction sequences within the source code slice to obtain the purified source code slices extracted based on the source code slicing criterion.

The reason for removing the source code statements without corresponding assembly instruction sequences in the source code slice is that these source code statements are generally macro definitions, file-inclusion instructions (e.g., *#include*), preprocessing instructions (e.g., *#undef*, *#line*, *#error*), function declarations, and variable declarations. These source code statements are already converted into preprocessing information during the compilation and preprocessing stages; therefore, no assembly instructions are generated when the processor executes the program. Removing such statements helps shorten the slice length and reduces the interference of vulnerability-unrelated statements in the model's learning of vulnerability patterns.

Taking the corresponding code shown in Figs. 1(a) and 1(b) as an example, the generation and purification processes of the bimodal code slices based on the source code slicing criterion are shown at the top of Fig. 3. Fig. 3(a) shows the AST and PDG generated from the source code. The bottom of Fig. 3(a) shows the source code-based program slice generated using the parameter n in Line 4 of the code shown in Fig. 3(a) as the slicing criterion. Fig. 3(b) shows the assembly code slices generated based on the source code slicing criterion. Fig. 3(c) shows the source code slice after the purification process. The nodes of an AST are a tree representation of the abstract syntax structure of the source code, where each node usually corresponds to a syntactic construct in the source code, such as expressions and statements. The leaf nodes of the AST are typically identifiers such as variable or function names or numbers, strings, and Boolean values. Vulnerability candidate key points are extracted from the leaf nodes in the AST, such as the AST leaf node function parameter n of Line 4 in Fig. 3(a), as a vulnerability candidate key point of the function parameter type. The function parameter n is the key point for vulnerability candidates. The relevant statements are then merged forward and backward in the PDG to obtain the unpurified source code slices at this point. The complete assembly code instruction sequences corresponding to the statements are mapped to the unpurified assembly code slices using the bimodal fusion information obtained in Section 4.1. Finally, the proposed source code slice purification method is employed to purify the source code slices. Lines 4 and 5 are not executed during the execution of the assembly instructions; thus, they are deleted from the source code slice to obtain the source code slice. After purifying the source code slices, the example is reduced from eight lines to six lines, specifically Lines 6, 9, 10, 11, 12, and 14 of the source code statements as well as the corresponding assembly code. This helps reduce the interference of vulnerability-unrelated statements in the source code with the model.

4.2.2. *Purification of the assembly code slices after bimodal cross-slicing*

To help the model understand the semantics of the assembly code, we merge the assembly code slices based on the source code slicing criterion and the assembly code slices based on the assembly code slicing criterion to form a complete assembly slicing dataset. To reduce parts of the assembly code that are not conducive to semantic understanding and vulnerability detection, we apply an assembly code slicing purification method to clarify the semantics of assembly instructions. In other words, we replace the register information that is not conducive to the semantic understanding of assembly code with variable/parameter information that is conducive to the semantic understanding of assembly code and delete other comment information. This helps include more easily understandable program semantics in assembly code slices to extract relevant semantic information more accurately from vulnerabilities. The specific steps are as follows:

First, we extract memory operation instructions related to the variables in the source code based on the assembly code that contains the

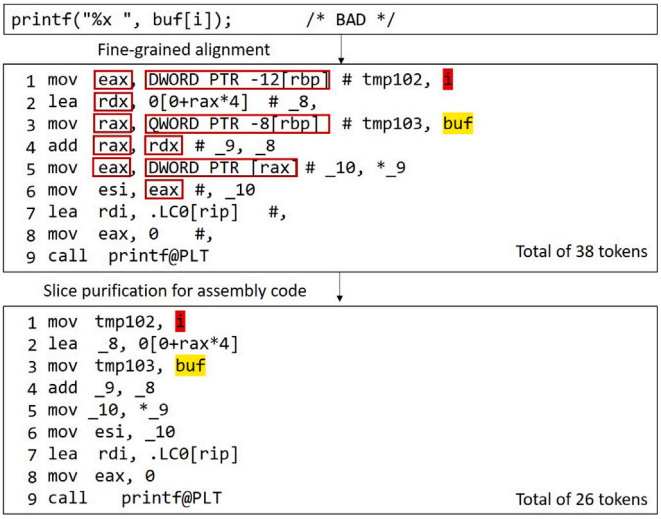

Fig. 4. Example of purification of assembly code slice.

bimodal fusion information obtained in Section 4.1. In other words, based on the assembly code slicing criterion, each variable is selected as a candidate vulnerability key point and is associated with all memory operation instructions in the assembly code. Second, according to the mapping relationship between the source code and the assembly code, we identify the source code statements corresponding to the memory operation instructions and obtain the source code slices based on the assembly code slicing criterion. The source code statements in these code slices contain the corresponding memory operation instructions; thus, the source code slices do not need to be purified. Third, we merge the purified source code slices generated based on the source code slicing criterion and the source code slices generated based on the assembly code slicing criterion to form the source code slices dataset. Fourth, based on the assembly code that contains the bimodal fusion information, the complete sequence of assembly instructions corresponding to each statement in the source code slices is identified, and the unpurified assembly code slices based on the assembly code slice criterion are obtained. Finally, the unpurified assembly code slices obtained in Sections 4.2.1 and 4.2.2 are merged and purified to form assembly code slice datasets, where the purification process refers to replacing the operand information of the assembly slice with the comment information in the fused assembly information.

Because operand information, such as register names, is introduced during compilation time and does not exist in the source code and register names vary among different assembly instruction sets, it is difficult for the model to learn the semantics of assembly code. Fortunately, the comments following the assembly instructions contain the names of the function parameters, variables from the source code, and temporary variables generated during compilation. This substitution helps the model to more accurately understand the semantics of the assembly code and extract semantic information related to vulnerabilities in the program.

Taking the statement in Line12 of Fig. 1(a) as an example, the assembly code containing bimodal fusion information and the purified assembly code slice corresponding to this statement are shown in Fig. 4. The purification rules for the assembly code slices are as follows:

- If all operands following an assembly operator have corresponding variable names in the comments of the assembly instruction (e.g., the assembly instructions in Lines 1 and 3), then all operands in the assembly instruction are replaced with the corresponding comments.
- If the length of the comment is less than the length of the operand, then the operands without the corresponding variable comments are retained, and only the operands with variable comments are replaced (e.g., the assembly instructions in Lines 2 and 6).
- If the instruction does not have a variable comment, the replacement operation is not performed (e.g., the assembly instruction in Line 7).

In the first assembly instruction in Fig. 4, the original two difficult-to-understand operands were replaced with the *mov* operation of variable *i* and temporary variable *tmp102*, in which the original difficult-to-understand semantics were changed to easily understandable semantics. As a result, the assembly code corresponding to one statement of source code was reduced from 38 to 26 tokens.

4.3. Vulnerability statement labeling

We merge the source code slices obtained in Sections 4.2 and label the statements in the bimodal slices. The statements with vulnerabilities are labeled as 1, and the other statements are labeled as 0 to indicate that they are "non-vulnerable."

4.4. Design of a bimodal fine-grained vulnerability detection network based on the Transformer architecture

This section describes the Transformer architecture of the proposed bimodal fine-grained vulnerability localization model. The network employs a Transformer encoder–decoder architecture. Unlike traditional classification models, the encoder–decoder architecture, which is used for sequence–sequence generation, implements sequence-to-sequence mapping. The Transformer model mainly consists of two parts: an encoder and a decoder. The encoder uses CodeBERT to encode the important semantic information contained in the input bimodal code sequence into vectors, whereas the decoder uses these vector representations output by the encoder to generate semantic vector representations for each statement using the self-attention mechanism. The statement vector representations output by the decoder are used as inputs to the vulnerability classifier to implement the vulnerability localization task.

4.4.1. Design of the encoder statement encoding network

Here, we describe how each bimodal statement within a bimodal code slice is input into the statement encoding network CodeBERT. As shown in Fig. 6, the vector representation of the fusion information for each statement is obtained. In the Transformer architecture, a series of steps is required for token embedding and statement encoding:

First, it is challenging to directly apply the Code-BERT layer as the statement encoding network because CodeBERT does not natively support the assembly code. Therefore, an extended CodeBERT vocabulary is constructed to better represent the statement vectors, instructions, and register tokens (e.g., '*mox*', '*rax*') in the assembly language. Second, the source and assembly code statements are tokenized by the CodeBERT tokenizer. Third, the tokens of the source code statements are placed between token *[CLS]* and token *[SEP]*, whereas the assembly instruction sequences of each source code statement in the purified assembly code slice are flattened into one line. The flattened assembly instruction sequences are then placed between token *[SEP]* and token *[EOS]*. Finally, the token sequence obtained earlier is input into the CodeBERT layer to output the statement vector representation after fusion of the bimodal features.

As shown in Fig. 6, a bimodal code slice contains a source code slice $C = [c_1, c_2, \ldots, c_i, \ldots, c_n]$ and an assembly code slice $A = [a_1, a_2, \ldots, a_i, \ldots, a_n]$, where c_i indicates that the statement in the source code slice i is the ith statement, a_i denotes the assembly instruction corresponding to the source code, and n represents the number of statements in the code slice. Further, for $c_i = [T_1, T_2, \ldots, T_S]$, T is the source code token

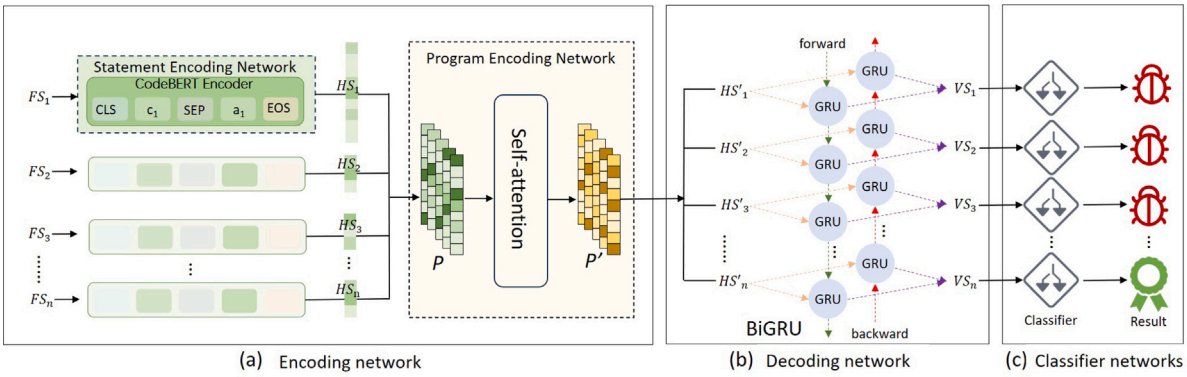

Fig. 5. Transformer-based bimodal fine-grained vulnerability detection network.

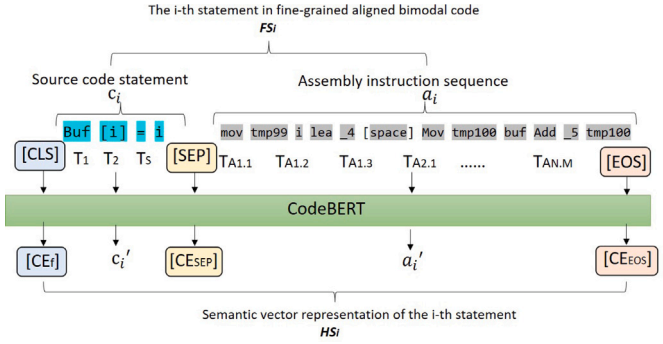

Fig. 6. The CodeBERT encoder for each statement encoding.

in the statement, S is the number of tokens within the statement. For $a_i = [TA_{1.1}, TA_{1.2}, TA_{1.3}, \ldots, TA_{N.M}]$, a_i is the sequence of assembly instructions corresponding to the source code statement c_i; N is the number of assembly instructions, where a single statement of the source code may correspond to several assembly instructions; TA s the assembly instruction token within a_i; and M is the number of tokens within the assembly instruction, which typically has a value of 3 for the X86 instruction set used in the experiments. The calculation formula is as follows:

$$FS_i = Concat(CLS, c_i, SEP, a_i, EOS)$$

where FS_i is the ith statement of the bimodal code arranged according to CodeBERT features and the proposed method, and is input into the statement encoding network CodeBERT. The calculation formula is as follows:

$$HS_i = CodeBERT(FS_i)$$

where HS_i is the hidden vector representation of the bimodal fusion information of the ith statement, which is output by the CodeBERT statement encoding network.$HS_i = [CE_f, c_i', CE_{SEP}, a_i', CE_{EOS}]$, where CE_f, CE_{SEP}, CE_{EOS}, c_i' and a_i' are the CLS, SEP, EOS, c_i and a_i outputs from CodeBERT.

As shown in Fig. 5(a), the vector representation of the statement fusion information in the bimodal code slice is obtained after processing through the statement encoding network $P = [HS_1, HS_2, \ldots, HS_i, \ldots, HS_n]$, where P is the hidden vector representation of a bimodal code slice obtained through the statement encoding network.

4.4.2. Design of the encoder program encoding network

In this section, we describe how a vector representation of the statement fusion information obtained from the statement encoding network for each statement in the bimodal code slice is input into the program encoding network to allow the model to learn the long dependencies of the statements. The specific steps are as follows:

The statement vector representation of the fused features within the bimodal code slice is output by the statement encoding network and then input into the program encoding network consisting of a self-attention mechanism to learn the long dependencies of vulnerability features between statements and obtain a feature-enhanced bimodal hidden vector representation. The statement encoding and program encoding networks together form the encoder of the Transformer architecture.

The hidden vector representation of the bimodal code slice is obtained through a self-attention-based program encoding network. The calculation formula is as follows:

$$Q = W^Q \cdot P$$
$$K = W^K \cdot P$$
$$V = W^V \cdot P$$
$$P' = Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_K}})V$$

where P' is the hidden vector representation of the bimodal code slice after the program encoding network. Specifically, $P' = [HS_1', HS_2', \ldots, HS_i', \ldots, HS_n']$, where HS_i' is the hidden vector representation of the statement i after processing through the encoding network.

4.4.3. Design of the decoder

As shown in Fig. 5(b), the decoder comprises BiGRU [12], which is used to learn the back-and-forth relationship of the statement sequences and enhance the structural features among the bimodal statement sequences. HS_i' is the hidden vector representation of the ith statement after processing through the encoding network, which is input into the decoder consisting of BiGRU. The output of the decoder is the semantic vector representation of statement. The calculation formula is as follows:

$$VS_i = BIGRU(HS_i')$$

where HS_i' is the hidden vector representation of the statement HS_i' obtained by the semantic vector representation from the decoder.

4.5. Model training

The semantic vector representation of each statement output by the decoder is input to the classifier to predict whether the statement is a vulnerability statement. The prediction result is represented as a vector whose number of vector components is equal to the number of source code statements. The index position value of each vector denotes the vulnerability prediction results of the corresponding position statement.

As shown in Fig. 5(c), the semantic vector representation obtained from the decoder is input into the classifier composed of fully connected

network layers to obtain the prediction result. The label "0" represents a normal line, whereas the label "1" represents a vulnerable line. The cross-entropy is calculated according to the predicted vector and the labeling information. The parameters of the bimodal representation learning models are adjusted according to loss backpropagation until the loss value no longer decreases. The vector representation of the ith statement (i.e., VSi) output by the decoder is input into the classifier, and the output of classifier ri is the vulnerability prediction result of the ith statement. The calculation formula is as follows:

$$r_i = FCN(VS_i)$$

The element r_i of $R = [r_1, r_2, \ldots, r_i, \ldots, r_n]$ is the fine-grained vulnerability localization result, where r_i is the classification result of the ith statement in the code slice R is the vector of classification results of the code slice, and n is the total number of input statements, which varies with the number of input statements.

4.6. Model inference

First, we generate assembly code that is fine-grained aligned with the source code to be detected and contains bimodal fusion information. Second, we use the bimodal cross-slicing and code slice purification methods to preprocess the bimodal code. Third, we employ a trained Transformer-based vulnerability localization model to detect vulnerability. Fourth, we collect statement localization results from each code slice and tag them to the corresponding statement in the source code. Finally, we export the fine-grained vulnerability detection report.

5. Evaluation settings

5.1. Research questions

In this section, we focus on the following research questions, to validate the proposed method.

- **RQ1:** How effective is the coarse-grained vulnerability detection of the proposed method, and is it better than the SOTA method?
- **RQ2:** How effective is the proposed method for fine-grained vulnerability detection on real-world project datasets of different sizes, and does it outperform the SOTA method?
- **RQ3:** How effective is the proposed method in detecting vulnerabilities of different Common Weakness Enumeration (CWE) types?
- **RQ4:** How does bimodal slicing purification affect vulnerability detection results?

5.2. Dataset

Because the proposed method requires experiments on both the source code and the assembly code, we must use full project data that can be successfully compiled. In previous work, we collected, preprocessed, and published the bimodal dataset, which is available at https://github.com/vulcoca/vul_coca. This bimodal dataset includes the bimodal SARD dataset and two bimodal real-world project datasets: version 1.2.2 of [13] and version 1.0.1e of [14]. FFmpeg version 1.2.2 contains 3478 source files and 637 test cases, and OpenSSL version 1.0.1e contains 2203 source files and 636 test cases.

For RQ1, we primarily used these two datasets in the proposed experiments to compare them with our previous work [5]. For RQ2, to validate the detection effectiveness of the proposed method on real-world project datasets of different sizes, we supplemented it with two more bimodal real-world project datasets: version 9.2.4 of [15] and version 1.8.3 of [16] (SVN). PostgreSQL version 9.2.4 contains 5458 source files and 637 test cases, and Apache Subversion (SVN) version 1.8.3 contains 1728 source files and 638 test cases. These datasets are available at https://github.com/alg4ahvnt/postg-svn. We used 80% of the dataset as the training set, and 20% was used as the testing set. For RQ3 and RQ4, we merged the bimodal datasets of the above four real-world projects for experimental analysis (see Table 1).

To ensure that code slicing more complete to cover vulnerabilities, we selected the following seven types of bimodal code vulnerability candidate critical nodes. For source code-based bimodal slicing, we selected six types of vulnerability candidate key points, including four types of the vulnerability candidate key points proposed in SySeVR [2]: API/library function calls, associated with vulnerabilities such as resource leakage, buffer overflow, memory management issues, SQL injection, and cross-site scripting (XSS); Array Use, associated with vulnerabilities such as buffer overflow, uninitialized array, and array out-of-bounds errors; Pointer Use, associated with vulnerabilities such as null pointer reference, double free, and uninitialized pointer; and Arithmetic Expressions, associated with vulnerabilities such as integer overflows and underflows, division-by-zero errors, loss of precision, and symbol errors. The two types of vulnerability candidate key points proposed by Vu1SPG [17] are function return, with possible associated vulnerability type such as unhandled error return value, memory leak, SQL injection, or XSS, and function parameter, with possible associated vulnerability types such as SQL injection, XSS, and formatting string. For assembly code-based bimodal slicing, we selected the vulnerability candidate key points proposed in our previous work [5], i.e., memory operations (MO), which are instructions and source code statements related to variable/parameter memory operations.

5.3. Ground-truth labels

For slice-level ground-truth labels, if there is one or more lines of vulnerability statements within the code slice, the code slice is marked as "1" to indicate that it contains vulnerabilities; otherwise, it is marked as "0" to indicate that it does not contain vulnerabilities.

For line-level ground-truth labels, we counted the number of statements within a slice and initialized a one-dimensional array of statements of the same length. We initialized all array element values to "0" to indicate that the statement is not a vulnerability statement and then identified the location of the vulnerability statement within the slice. We modified the array element at the corresponding location to "1" to indicate that the statement is a vulnerability statement.

5.4. Baseline methods

5.4.1. Methods for coarse-grained vulnerability detection

For the coarse-grained vulnerability detection task, we used the following four baseline methods for experimental comparison: vulnerability detection tools based on pattern matching [18,19], deep learning-based token-based methods (e.g., Sy-SeVR [19]), GNN-based methods (e.g., IVDetect [20]), and bimodal-based methods (e.g., VDTC [5]).

5.4.2. Methods for fine-grained vulnerability detection

For the fine-grained vulnerability detection task, we used the following two baseline methods for experimental comparison: VulDeelocator [21], which directly localizes vulnerability statements using an attention mechanism, and LineVul [6], which is a Transformer-based method.

5.5. Parameter configurations

The model hyperparameters used in this study (Table 2) provide the best experimental results that can be obtained when using these values.

Table 1
Number of code slices for different datasets.

DATASET	#Total lines	#Avg lines per file	#Slices	#Vulnerability slices	#Vulnerability lines
SARD	1 978 705	91	163 726	48 648	10 160
FFmpeg	1 040 679	1634	15 486	2242	8299
OpenSSL	358 518	564	50 056	4679	17 314
PostgreSQL	898 075	1410	62 892	3618	14 702
SVN	571 069	895	3373	1124	3829

Table 2
Hyper-Parameter Value.

Hyper-parameter	Values	Descriptions
CodeBERT embedded dimension	768	Using the CodeBERT-large model
Dropout	0.1	Dropout value
Batch size	20	Number of bimodal code slices in a Batch
Epoch	early-stopping	Loss value not decreasing after 5 epochs
learning-rate	5e−5	Learning rate
optimizer	Adam	Adaptive Moment Estimation
Vocab Size	85	Number of CodeBERT vocabularies populated with compilation tokens

5.6. Experimental environment

The experimental environment for this work was as follows. The hardware was a desktop computer with an AMD thread ripper 3960X CPU, 32 GB RAM, and GeForce RTX 4090D GPU. The software included Ubuntu 20.04 LTS, PyTorch version 2.2.1 (stable), CUDA 11.8, GCC version 7.5 C/C++ compiler, GDB version 7.6.1 C/C++ debugger, and GMP header version 6.1.2.

5.7. Evaluation metrics

To evaluate the performance of the proposed method for coarse-grained vulnerability detection tasks, we used the following common evaluation metrics:

The metric accuracy measures the correctness of all detected vulnerabilities. Accuracy (A) is defined as follows:

$$A = \frac{TP + TN}{TP + FP + TN + FN}$$

The metric precision measures the proportion of truly vulnerable samples among the detected (or claimed) vulnerable samples. The precision (P) is defined as follows:

$$P = \frac{TP}{TP + FP}$$

The metric recall measures the proportion of true-positive samples among the total number of vulnerable samples. Recall (R) is defined as follows:

$$R = \frac{TP}{TP + FN}$$

The metric F1-measure measures the overall effectiveness of the proposed method by considering both the precision and the false-negative rate. The F1-measure (F1) is defined as follows:

$$F1 = \frac{2PR}{P + R}$$

The metric false-positive rate (FPR) measures the proportion of false-positive samples among the nonvulnerable samples. The FPR is defined as follows:

$$FPR = \frac{FP}{FP + TN}$$

The metric false-negative rate (FNR) measures the proportion of false-negative samples among the vulnerable samples. The FNR is defined as follows:

$$FNR = \frac{FN}{TP + FN}$$

where TP is true positives, FP is false positives, FN is false negatives, and TN is true negatives.

Table 3
The coarse-grained vulnerability detection results on the SARD dataset.

Method	A(%)	P(%)	R(%)	F1(%)	FPR(%)	FNR(%)
Flawfinder	61.6	48.9	16.3	24.5	10.5	83.7
Checkmarx	39.1	27.9	45.5	34.6	72.8	54.5
SySeVR	89.0	84.2	87.8	86	10.1	12.2
IVDetect	96.8	96.3	88.6	92.3	0.9	11.4
VDTC	97.4	92.1	**94.8**	93.4	2	**5.2**
Our Method	**97.4**	**96.7**	92.7	**94.7**	**0.1**	7.3

These three metrics are specific to the use of fine-grained vulnerability localization tasks.

The IOU is a specialized metric for fine-grained vulnerability localization that can reflect the vulnerability localization accuracy of the model. The IOU is calculated as follows:

$$IOU = |V \cap U| / |V \cup U|$$

where V is the predicted vulnerability statement location and U is the true vulnerability statement location.

In fine-grained vulnerability detection experiments, FNR and FPR were also used as metrics; however, unlike coarse-grained metrics, these two metrics are calculated based on detected vulnerability statements rather than detected vulnerability samples.

6. Evaluation results

6.1. Experiments to answer RQ1

The purpose of this experiment was to validate the performance of the proposed method in a coarse-grained vulnerability detection task. We compared the proposed method with the baseline methods introduced in Section 5.4 on the SARD dataset and two real-world project datasets, and their experimental results are shown in Tables 3 and 4, respectively. As shown in Tables 3 and 4, the proposed vulnerability detection method based on the Transformer architecture and bimodal code feature fusion achieved the best performance, with an FPR of only 0.1%–0.2%, primarily due to the following reasons. First, the fine-grained aligned bimodal information is fully utilized to capture vulnerability features using the Transformer-based model, which makes a better use of the fine-grained alignment information than in previous studies. Second, the Transformer architecture is used to extract the vulnerability features of the bimodal code and fuse them at the statement level, which enables accurate and effective extraction of vulnerability-related features. Finally, a staged code representation based on statement encoding and program encoding networks is used to capture the relationships within and between statements, enabling it to better handle the long-term dependency problem.

As shown in Table 3, the two pattern matching-based detection tools, Flawfinder and Checkmarx, have either high FPR or high FNR, primarily because the vulnerability detection results of these tools rely heavily on the predefined vulnerability pattern libraries, making it difficult to identify new or complex vulnerability types. However, compared with the pattern matching-based methods, the overall performance improvement of vulnerability detection using SySeVR was very limited because it only uses Syntax-based Candidate Vulnerabilities (SyVCs) and Semantic-based Candidate Vulnerabilities (SeVCs) extraction, which are not sufficient to capture the vulnerability features. Compared with SySeVR, the overall performance, especially accuracy, of IVDetect, a GNN-based deep learning method, was significantly improved because it employs five intermediate code representations, each of which reveals vulnerability features from a different perspective of the code. Such multiperspective analysis helps improve the accuracy of vulnerability detection. Although IVDetect can extract rich vulnerability features from the source code, it cannot capture the vulnerability features contained in assembly code. The method we previously proposed, VDTC, uses a coattention mechanism to learn cross-modal enhancement and fusion features from the source and assembly code, which further improves the overall performance of slice-level vulnerability detection. However, VDTC uses only slice-level bimodal alignment information to extract vulnerability features, and the fine-grained alignment bimodal information is not fully utilized; therefore, the overall performance of VDTC is slightly lower than that of the proposed method.

Table 4 compares the vulnerability detection results of the proposed method and two other baseline methods (i.e., IVDetect and LineVul) on two real-world bimodal datasets (i.e., FFmpeg and OpenSSL). The first baseline method, IVDetect, employs context-aware representation learning to obtain the vector representation of each statement. Then, it uses these learned vectors for vulnerability detection using FA-GCN (feature-attention graph convolution network). The attention-based BiGRU model is used to learn the vector representation of each statement based on the contextual code surrounding the vulnerable statements. The limitations of BiGRU networks in handling long-term dependencies, such as gradient vanishing or exploding issues when handling very long token sequences, limit the overall performance improvement of IVDetect despite using five intermediate code representations to extract vulnerability features. The second baseline method, LineVul, uses a Transformer-based framework consisting of 12 Transformer encoders with a bidirectional self-attention mechanism and can effectively capture long-term dependencies and semantics in the source code. The self-attention mechanism of the Transformer model allows it to consider information from all other tokens in the sequence when processing each token without being limited by the sequence length. This makes LineVul effectively capture the meaningful long-term dependencies and semantics of the source code, thereby solving the gradient vanishing or explosion problem that RNN-based architectures may encounter, resulting in high recall and high F1-measure. However, the proposed method achieved a lower FPR than these two baseline methods, which may be due to the use of fine-grained aligned bimodal code information in addition to the Transformer-based architecture, which helps improve the accuracy of vulnerability detection. High accuracy helps reduce the manual confirmation workload.

Conclusion: In the coarse-grained vulnerability detection task, the proposed method effectively captured vulnerability features, achieving a large improvement in FPR reduction.

6.2. Experiments to answer RQ2

The purpose of this experiment was to validate the performance of the proposed method in the fine-grained vulnerability detection task. To make the experiment more comprehensive, we supplemented our study with two real-world project datasets of different sizes and complexities: PostgreSQL and Apache Subversion (SVN). PostgreSQL is a powerful open-source object-relational database management system, and Apache Subversion (SVN) is a popular code-version control system. As shown in Table 5, the four datasets are maintained by different organizations, vary in size and characteristics, belong to completely different software application areas, and exhibit different coding styles, making the dataset vulnerability types more comprehensive. The number of slices reflects the size of the datasets, and the average number of lines in a slice reflects the code dependency of the datasets. The average number of lines in a function reflects the complexity of the datasets because the number of lines in the source code can serve as a reference for measuring software complexity. The number of assembly instructions corresponding to each code statement reflects the complexity of the compiled code. From these observations, we found that the PostgreSQL dataset was the largest, with 62,892 slices and an average number of function lines of 33. The newly added SVN dataset had the largest number of lines of code in a single slice, which suggests a larger PDG from the SVN conversions. In addition, the average number of lines of code per slice was 278, suggesting that the project may contain more complex functionality. OpenSSL's relatively small average number of lines of code in a slice implies that its functionality is more modular. The high number of assembly instructions per line of code in FFmpeg may indicate several low-level operations or optimizations in the source code. These characteristics may make vulnerability detection more difficult.

Table 4
Coarse-grained vulnerability detection experiment results on FFmpeg and OpenSSL datasets.

Method	Dataset	A(%)	P(%)	R(%)	F1(%)	FPR(%)	FNR(%)
IVDetect	FFmpeg	89.7	94.7	83.3	84.0	7.3	16.7
	OpenSSL	92.9	89.9	88.3	89.1	4.8	11.7
	Average	91.3	87.2	85.8	86.6	6.1	14.2
VDTC	FFmpeg	95.2	87.9	89.2	88.5	3.3	10.8
	OpenSSL	95.6	87.7	91.8	89.7	3.4	8.2
	Average	95.4	87.8	90.5	89.1	3.4	9.5
LineVul	FFmpeg	95.6	93.8	92.1	92.9	2.7	7.9
	OpenSSL	95.6	93.8	94.3	94.0	3.6	5.7
	Average	95.6	93.8	**93.2**	**93.4**	3.1	**6.8**
Our Method	FFmpeg	99.2	95.4	88.1	91.7	0.2	11.9
	OpenSSL	99.2	93.4	95.1	94.3	0.2	4.9
	Average	**99.2**	**94.4**	91.6	93.0	**0.2**	8.4

From the experimental results in Table 6, we draw the following conclusions:

The first baseline method, VulDeeLocator, is the c However, its average IOU for the four real-world projects was only 49.1%. This is because VulDeeLocator still employs a coarse-grained cross-entropy function to calculate losses, making it difficult to provide accurate adjustment signals to guide model training, resulting in low generalization performance. The second baseline method, LineVul, is the SOTA method used for fine-grained vulnerability detection, and it is more accurate and efficient than existing vulnerability detection methods. LineVul solves the problem of RNN-based models struggling to capture long-term dependencies within a long sequence by using a BERT architecture with self-attention layers to generate code representation, resulting in an average IOU improvement of 28.9% compared with that of VulDeeLocator. However, the FPR of LineVul remained very high. Compared with LineVul, the proposed method achieved the best average IOU performance, especially with the FPR performance being more significant, and it did not exceed 0.2% on all datasets. This is primarily due to the proposed method effectively using statement-level fine-grained bimodal alignment and purified sliced code, which can capture fine-grained vulnerability features from multiple modalities and enhance the model's understanding of the semantics of assembly code, avoiding interference from irrelevant vulnerability information. In addition, using the Transformer architecture consisting of statement encoding and program encoding networks to implement staged representation learning of code also enables it to accurately and effectively

Table 5
Detailed information of the real-world project datasets we used.

Dataset	#Slices statements	#Avg statements	# Max instructions	# Avg statements/function	#Avg/statement
FFmpeg	15 486	29	136	26	136
OpenSSL	50 056	14	182	16	56
PostgreSQL	62 892	31	133	33	104
SVN	3373	33	278	19	120

Table 6
Experimental results of fine-grained vulnerability detection on real-world project dataset.

Method	Dataset	FPR(%)	FNR(%)	IOU(%)
Vuleelocator	FFmpeg	64.1	18.4	61.4
	OpenSSL	22.3	37.7	56.2
	PostgreSQL	43.2	66.9	32.1
	SVN	42.8	46.1	46.9
	Average	43.1	42.2	49.1
LineVul	FFmpeg	29.4	20.9	72.7
	OpenSSL	48.3	6.6	87.9
	PostgreSQL	40.0	19.9	74.1
	SVN	38.3	15.1	77.6
	Average	39.0	15.6	78.0
Our method	FFmpeg	0.1	9.6	83.9
	OpenSSL	0.1	10.7	86.7
	PostgreSQL	0.1	3.4	82.3
	SVN	0.2	8.7	85.1
	Average	**0.1**	**8.1**	**84.5**

capture long-term dependencies and vulnerability features of code with complex structures.

Conclusion: In the fine-grained vulnerability localization task, the proposed method achieved good detection results on a wider range of vulnerability datasets with different coding styles and program functions.

6.3. Experiments to answer RQ3

The purpose of this experiment was to count the types of vulnerabilities in the datasets and determine how the proposed method performs on different CWE types. Table 7 lists the 20 CWE vulnerability types and their sample sizes based on their frequency of occurrence in the real-world project dataset we used. Similar CWE types can be categorized within the same horizontal line. CWE-476, CWE-78, CWE-89, CWE-416, and CWE-190 are listed in MITRE's CWE Top 25 for 2021, based on the 2019 and 2020 Common Vulnerabilities data published by the NVD.

RQ3 was tested on a single CWE type, and all tested codes were positive. The statement-level recall metric was employed in the evaluation. We compared the source code-based method LineVul with the proposed bimodal-based method. The statement-level recall of each CWE vulnerability type is illustrated in Fig. 7. As shown in Fig. 7, the features of injection-type vulnerabilities, such as CWE-88, CWE-78, CWE-89, and CWE-129, were not obvious at the assembly code level but were obvious at the source code level. The source code contains variable names, function names, and other elements that directly reflect the program's intention and logical structure. Injection vulnerabilities, such as SQL injection vulnerabilities, typically involve string splicing to form a database query. The proposed method achieved high recall comparable to that of LineVul because it is a detection method based on source and assembly code and can extract useful features from the source code to detect this type of vulnerability.

For overflow vulnerabilities, such as CWE-124, CWE-805, CWE-190, and CWE-191, their features were obvious at the assembly code level but not at the source code level. The average recall metric of the proposed method exceeded that of the baseline method. The main reasons for this are as follows. The assembly language provides direct control over MO, including loading, storing, and moving data. Overflow vulnerabilities typically involve the mismanagement of memory boundaries, which can be observed directly in the assembly code. Register usage, including registers used for specific operations, is clearly visible in the assembly instructions. For example, the integer overflow vulnerability CWE-190 involves data transfers between registers and arithmetic operations, which are more easily identified in the assembly code. The overflow vulnerability may affect the control flow of the program, and the order of execution of the instructions and jump logic are apparent; thus, it is easier to identify potential overflow points.

The features of CWE-476, CWE-400, and CWE-401 were obvious in the assembly code and were usually caused by programming logic errors, such as uninitialized pointers and incorrect memory management. These logic errors can be easily found in assembly code. The proposed method achieved recall metrics of 95.2% for these types of vulnerability.

CWE-416 and CWE-789 are types of vulnerabilities related to memory management and usage. The proposed method achieved an average recall of 99.2%, far exceeding the baseline method, which is mainly attributed to the assembly code providing detailed and specific information about the data flow. The assembly code corresponds directly to machine instructions; thus, it is possible to demonstrate the precise movement of data between registers and memory, which allows us to better capture vulnerability features.

The detection performance of the proposed method for vulnerabilities such as CWE773 and CWE775 was far superior to that of the baseline method, primarily because CWE773 and CWE775 are both vulnerabilities related to resource handles, and their features are suitable for capturing from bimodal code. In source code, resource allocation and release are usually managed by abstract structures in high-level languages. These structures are easily identified in source code and can help us understand the life cycle of a resource. In addition, the manipulation of handles, including creation, use, and destruction, can be observed directly in assembly code; thus, it is easy to analyze vulnerabilities and track the application and destruction of handles.

The proposed method performed the worst in terms of detecting CWE-367 vulnerabilities among all vulnerability types. CWE-367 is a type of vulnerability known as the time-of-check to time-of-use race condition. This occurs when a program checks the status of a resource, but the status changes before the resource is used. Because the cause of this vulnerability depends on external factors, both the proposed and baseline methods exhibited unsatisfactory detection performance for this type of vulnerability.

Conclusion: Overall, the proposed bimodal-based fine-grained vulnerability detection method outperformed the source code-based vulnerability detection methods, with 100% recall for the CWE-416 vulnerability type.

6.4. Experiments to answer RQ4

To compare the impact of our proposed bimodal-based slice purification method on the detection results, we performed ablation experiments with and without the slice purification method. As shown in Table 8, if the bimodal-based slicing purification method is used, the average IOU can be increased by 17.5%, especially on the SVN dataset, where the IOU was increased by 32.5%. This indicates that the bimodal-based slicing purification method can more accurately predict the line where the vulnerability statement is located, thereby reducing the FPR and FNR of the vulnerability statement.

Table 7
The top 20 most frequently occurring CWE types and their sample sizes in the real-world project dataset.

CWE type	Vulnerability description	#Vulnerability Statements
CWE-88	Argument Injection	575
CWE-78	OS Command Injection	562
CWE-89	SQL Injection	443
CWE-129	Improper Validation of Array Index	185
CWE-191	Wrap or Wraparound	212
CWE-190	Integer Overflow or Wraparound	62
CWE-195	Signed to Unsigned Conversion Error	173
CWE-805	Buffer Access with Incorrect Length Value	140
CWE-124	Buffer Underflow	102
CWE-127	Buffer Under-read	681
CWE-400	Uncontrolled Resource Consumption	785
CWE-401	Missing Release of Memory after Effective Lifetime	144
CWE-367	Time-of-check Time-of-use Race Condition	224
CWE-363	Race Condition Enabling Link Following	71
CWE-773	Missing Reference to Active File Descriptor or Handle	141
CWE-775	Missing Release of File Descriptor or Handle after Effective Lifetime	142
CWE-416	Use After Free	77
CWE-789	Memory Allocation with Excessive Size Value	61
CWE-476	NULL Pointer Dereference	1203
CWE-835	Infinite Loop	55

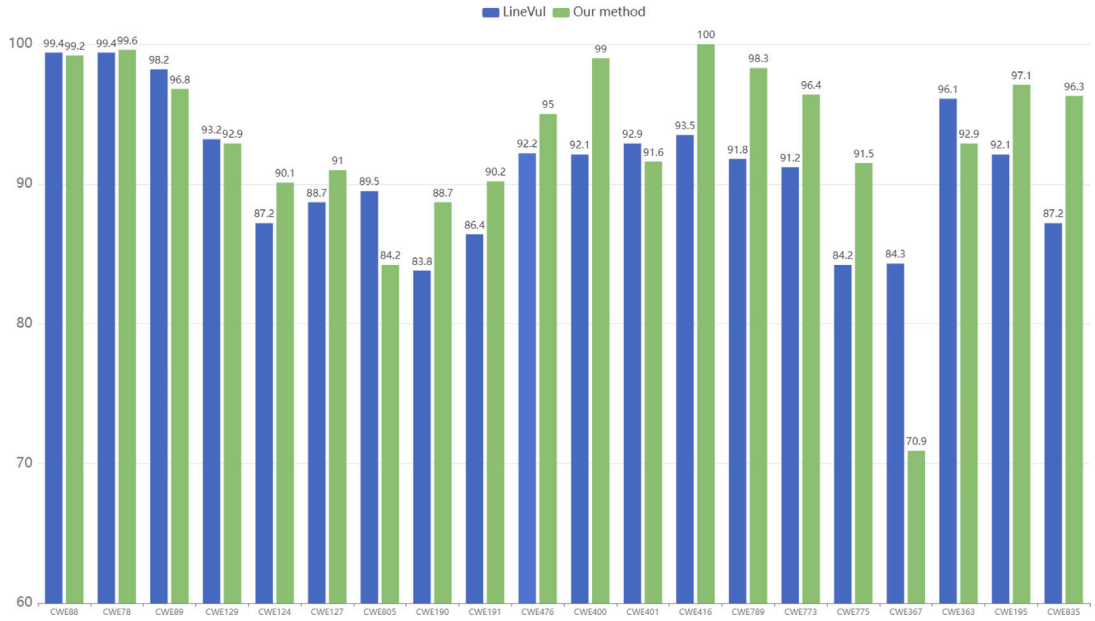

Fig. 7. The recall (i.e. 100%-FNR) at statement level for fine-grained detection on each CWE type.

Table 8
Comparison of fine-grained detection performance without and with using bimodal base slice purification on 4 real-world project datasets.

Method	Dataset	FPR(%)	FNR(%)	IOU(%)
Without	FFmpeg	0.2	27.4	69.3
	OpenSSL	0.5	18.2	76.0
	PostgreSQL	0.8	2.5	70.0
	SVN	0.1	38.6	52.6
	Average	0.4	21.7	67.0
With	FFmpeg	0.1	9.6	83.9
	OpenSSL	0.1	10.7	86.7
	PostgreSQL	0.1	3.4	82.3
	SVN	0.2	8.7	85.1
	Average	**0.1**	**8.1**	**84.5**

As shown in Table 9, the first column content reflects the total number of slices. Using the proposed bimodal-based slice purification method, the number of code slices was reduced from 134,167 to 121,807, and all source code statements in the reduced 12,360 slices did not have corresponding assembly instructions. Because source code statements without corresponding assembly instructions are not executed and cannot be related to vulnerabilities, removing them helps the purified code slice focus on potentially vulnerable statements and improves model learning efficiency. The second column in Table 9 shows the average length of source code slices, which decreased from 36 to 31 after applying the proposed method, indicating that the proposed method can reduce the length of source code slices, decreasing the model's focus on vulnerability-unrelated lines and accelerating the model's convergence. The third column in Table 9 lists the average instruction tokens in the assembly code gadgets, whose number was reduced from 260 to 182.

As shown in Table 10, for CodeBERT, after using the bimodal code slice purification method, the preprocessing time was reduced from 8.7 to 7.7 h, the training time for a single epoch was reduced from 8 to 5 min, and the number of training epochs was reduced from 70 to 30 epochs. These results indicate that the speed of convergence of the

Fig. 8. Two examples of fine-grained vulnerability detection.

Table 9
Comparison of the number of source code statements and the number of assembly tokens in slices without and with using the bimodal cross-slicing method.

Method	# slices	#Average of lines in source code slices	# Average of assembly tokens
Without	134 167	36	260
With	121 807	31	182

Table 10
Comparison of storage and time usage without and with using the bimodal slicing purification method.

Method	Preprocessing time	Training time	Training epoch	Total time	Storage usage
Without	8.7 h	8 m	70	18 h	15 GB
With	7.7 h	5 m	30	10.2 h	9 GB

model was greatly improved. Moreover, after using the slice purification method, the total training time was reduced from 18 to 10.2 h, and the consumed storage was reduced from 15 to 9GB, which significantly reduced the training time and further optimized the resource usage.

Conclusion: These results imply that the proposed bimodal-based slice purification method not only improves the performance of the model and reduces the time and resources required to train the model, ultimately improving the learning efficiency of the model.

7. Discussion

The top panel of Fig. 8 shows a code gadget from the *bio_err.c* source file in the OpenSSL project, which we refer to as Code Gadget 1. This file handles error functions related to the basic input/output system as part of OpenSSL's error-handling mechanism. The bottom panel of Fig. 8 shows a code gadget from the *e_camellia.c* file, which we refer to as Code Gadget 2. As part of the OpenSSL project, this file implements the Camellia encryption algorithm, which is used in many scenarios and is also critical to the project as a whole. We highlighted the vulnerability statement in Fig. 8 with a pink background, and the vulnerability probability output by the proposed method was labeled at the end of each vulnerable statement with a font color similar to that of the source code statements. Because of limited space, we highlight only the source code lines corresponding to the assembly instruction sequences to demonstrate that similar code of the corresponding assembly instructions are connected by the solid line.

Code Gadget 1 contains CWE-805 "Buffer Access with Incorrect Length Value." This vulnerability occurs when a program accesses an incorrect buffer.Transformer-based statement-level vulnerability detection captures cross-modal fine-grained features, thereby identifying potential buffer overflow issues. In Line 217, *strncpy()* is used to copy a string, which could lead to buffer overflow if the target buffer is not large enough. In Lines 218–220, the for-loop iterates over the buffer and converts characters to uppercase. If the buffer is not null, the for loop may be read beyond the buffer's end. In Line 222, the buffer is printed. If the buffer has overflowed, this may result in the printing of unintended memory contents. Combined with the assembly code on the right, the problem arises because the length returned by *strlen()* and stored in the rax register is used directly in *strncpy()* without checking whether it exceeds the bounds of the target buffer. This can lead to buffer overflow because *strncpy()* does not check the actual size of the target buffer and only copies the characters based on the provided length parameter. Hence, in the fusion information, the assembly code is a unique flag *tmp278* (i.e., temporary variable 278), and the subsequent for loop corresponds to the assembly code instruction blocks. *.L51* and *.L52* iterate over the buffer and call *stonesoup_toupper()* to convert the characters. If the buffer is not terminated correctly, this loop may access memory outside the buffer, thereby further increasing the security risk. Observation of the fusion information of the fine-grained aligned source and assembly code enables better detection of vulnerability features, which allows the model to capture vulnerability information more accurately.

Code Gadget 2 contains CEW-590 "Free of Memory Not on the Heap." In Line 164, the $free()$ function is free memory allocated on the heap; however, $buff$ is actually an array that cannot be freed using $free()$, which could cause a program crash. From an assembly code perspective, mapping the *rbp* register of buff, which is used to store the base address of the stack frame, conflicts with the *call* free instruction. The LineVul method produced false positives for this vulnerability. Lines 639, 641, and 646, similar to Lines 220, 222, and 217 in the previous Code Gadget 1, are not vulnerability lines; however, LineVul

produced false negatives and classified them as vulnerabilities. The proposed method accurately captures the fine-grained vulnerability features of source and assembly code; thus, it does not predict similar lines of code as vulnerability statements, effectively reducing false positives and false negatives.

8. Related work

8.1. Coarse-grained vulnerability detection

Coarse-grained vulnerability detection attempts to determine whether a function or code snippet is likely to contain vulnerabilities. In particular, coarse-grained vulnerability detection can be further divided into function-level and slice-level vulnerability detection.

Function-level vulnerability detection Zhou et al. [22] proposed a general GNN-based model for graph-level classification via learning on a rich set of code semantic representations. Building on this foundation, Wang et al. [23] extended the standard GNN using an enhanced GGNN to model multiple code relationship edges in a graph. To emphasize the classification separation between secure and vulnerable code, Wang et al. [24] exploited the class-separation features of statement types, post dominance trees, and exception flow graphs at different abstraction levels.

Slice-level vulnerability detection Li et al. [25] is the first to use deep learning-based vulnerability detection to relieve human experts from the tedious and subjective task of manually defining features. This work views the source code as a sequence of tokens, making it difficult to capture the structural characteristics of the code. In recent years, researchers have found that single-modal vulnerability detection models are unable to effectively extract cross-modal features. This limitation hinders the model's ability to fully capture the diverse information present in different data representations, reducing the overall detection performance. Hence, to address this issue, Li et al. [26] proposed a solution that integrates vulnerability features from both the source code and assembly code levels at the program slicing granularity. Tao et al. [5] proposed a new multi-modal deep learning based vulnerability detection method through a cross-modal feature enhancement and fusion. Multi-modal vulnerability detection techniques have become one of the popular research directions in the field of vulnerability detection for the future. Method by [27] determines the value-flow path that may have vulnerabilities and then gets the important vulnerability words based on the indexing of the attention weight of the statement.

Li et al. [25] was the first to apply deep learning-based vulnerability detection to relieve human experts from the tedious and subjective task of manually defining features. Their study viewed the source code as a sequence of tokens, which made it difficult to capture the structural characteristics of the code. In recent years, researchers have found that single-modal vulnerability detection models cannot effectively extract cross-modal features. This limitation hinders the model's ability to fully capture diverse information in different data representations, which reduces the overall detection performance. Therefore, to address this issue, Li et al. [26] proposed a solution that integrates vulnerability features from both the source and assembly code levels at the program slicing granularity. Tao et al. [5] proposed a multimodal deep learning-based vulnerability detection method that enhances and fuses cross-modal features. Multimodal vulnerability detection techniques have become a popular research direction in the field of vulnerability detection. The method described in Cheng et al. [27] determines the value-flow path that may have vulnerabilities and then obtains important vulnerability words based on indexing the attention weight of the statement.

8.2. Fine-grained vulnerability detection

Fine-grained vulnerability detection can identify specific lines of code that trigger vulnerabilities.

Methods based on source code: The edge regression-based method in [28] combines target detection methods using edge regression to achieve vulnerability localization. DeepLineDP [29] is a method that combines BGRU with a hierarchical attention network to learn the semantic attributes of surrounding tokens and lines automatically, which helps identify vulnerable files and lines. LineVul [7] employs the attention mechanism of the BERT architecture to perform line-level vulnerability prediction. LineVD [8] employs CodeBERT to generate vector representations at both function and statement levels, employing GAT to capture dependencies and jointly learning function-level and statement-level information to enhance detection performance. To better use the information of code-related graphs, SedSVD [30] uses code property graphs to learn the semantic and syntactic information of the source code. It selects central nodes in the CPG to construct subgraphs and applies a relation graph convolutional network (RGCN) to handle different edges distinctly.

Methods based on LLVM-IR: VulDeeLocator [21], which adapts to the semantic relationships between types and macro definitions, accurately captures control and data flows and introduces a fine-grained approach to achieve high accuracy using intermediate code. Similarly, the method in [31] performs fine-grained slicing of programs based on LLVM, labels vulnerability locations to obtain line-level slices of vulnerable programs, and uses BGRU to learn long-term dependencies between code slices.

Methods based on assembly code: VULOC [32] compiles and disassembles binary programs, leveraging the mapping between assembly and source code line numbers, combined with deep learning, to identify vulnerabilities. The current research on vulnerability localization techniques primarily focused on source code-based methods, and relatively few studies were based on intermediate and assembly code.

8.3. Vulnerability detection based on LLM

Previous studies [33,34] realized the task of vulnerability detection and localization by fine-tuning the large language model (LLM). A previous study [35] demonstrated LLM-based software vulnerability detection with graph structure and in-context learning. The authors in [36] proposed a novel LLM-based vulnerability detection technique, VulRAG, that leverages a knowledge-level retrieval-augmented generation framework to detect vulnerabilities in given codes in three phases. Fine-tuned LLMs performed weaker than the Transformer-based methods but were comparable to the GNN-based methods [37]. In addition, LLMs in the few-shot setting demonstrated lower performance than existing methods.

9. Threats to validity

9.1. Threats to internal validity

The internal threat comes from experimental bias and is related to the implementation of the comparison method and the dataset we used. To avoid experimental bias, we conducted our experiments using the publicly available implementation code of SySeVR, LineVul, IVDetect, and VulDeeLocator with the addition of our metrics. Because of the need for complete project source code that can be successfully compiled to build the bimodal vulnerability dataset, the existing publicly available vulnerability datasets are not suitable for our experiments. Therefore, we used the SARD dataset with the vulnerability database published by NDV and the complete project at GitHub. For the SOTA methods (e.g., SySeVR and LineVul), we used all codes from the original literature releases. To eliminate the realization bias, we double-checked the realizations and experiments to reduce the threat of comparison.

9.2. Threats to external validity

External threats are related to the generalizability of the proposed method. First, to evaluate the experimental results more comprehensively, we validated the proposed method on the SARD dataset and real-world open-source projects. However, we do not know how well it works for other CWE types because the dataset does not cover all CWE types. In addition, there are fewer data samples for some types, and the test samples for some CWE types (e.g., CWE833, 590, 831, and 822) may not be sufficiently rich, which may also affect the generalization performance of the model. Second, because the proposed method needs to use a compiler to compile the source code into assembly code, the tested code must contain complete file information (e.g., header files) and compile successfully. We used GCC version 7.5, the C/C++ debugger GDB version 7.6.1, GMP header version 6.1.2, and a unified compilation parameter to obtain the assembly code. The Clang compiler is a C language family frontend for LLVM based on LLVM-IR, and it is similar to JAVA bytecode. However, it does not output the assembly code, and we have not yet found a suitable alignment method. Therefore, it is not available for now. Third, we recommend that the source code of the training and test data be compiled into assembly code using uniform compilation parameters and optimization levels. Different compilation parameters and optimization levels can introduce variations in the assembly code, potentially affecting the experimental results. For example, if -O1 optimization is applied to the training data and -O3 optimization is used in the test data, discrepancies may arise. Finally, regarding the GCC version, we used GCC 6.0 or later versions without any issues. As with the earlier GCC version, we did not perform testing.

10. Conclusion

In this paper, we propose a new vulnerability detection method that excludes vulnerability-unrelated lines of source code and reduces the difficulty of semantic understanding of assembly code by fusing information from fine-grained aligned source and assembly code and by using a bimodal cross-slicing method in combination with a bimodal-based slice purification method. The proposed model is based on the Transformer architecture and analyzes the context of bimodal vulnerability code to capture cross-modal fine-grained vulnerability features at the statement level, thereby improving fine-grained vulnerability localization performance. In addition, it significantly reduces false positives and false negatives and improves the performance of fine-grained vulnerability localization tasks. A limitation of the proposed method is that it requires the source code to contain complete file information to successfully compile and generate assembly code aligned with the source code statements. This restricts validation to the bimodal dataset. In future, we plan to develop a more general cross-language vulnerability detection method to further improve the detection performance.

CRediT authorship contribution statement

Wenxin Tao: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Xiaohong Su:** Writing – review & editing, Writing – original draft, Visualization, Methodology, Funding acquisition, Formal analysis. **Yekun Ke:** Validation, Software, Project administration. **Yi Han:** Software, Resources, Project administration, Methodology. **Yu Zheng:** Visualization, Validation, Software. **Hongwei Wei:** Writing – review & editing, Visualization, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] S. Liu, G. Lin, L. Qu, J. Zhang, O. De Vel, P. Montague, Y. Xiang, CD-vuld: Cross-domain vulnerability discovery based on deep domain adaptation, IEEE Trans. Dependable Secur. Comput. 19 (1) (2020) 438–451.

[2] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, Sysevr: A framework for using deep learning to detect software vulnerabilities, IEEE Trans. Dependable Secur. Comput. 19 (4) (2021) 2244–2258.

[3] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, H. Jin, Vulcnn: An image-inspired scalable vulnerability detection system, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2365–2376.

[4] C. Zhang, Y. Xin, VulGAI: vulnerability detection based on graphs and images, Comput. Secur. 135 (2023) 103501.

[5] W. Tao, X. Su, J. Wan, H. Wei, W. Zheng, Vulnerability detection through cross-modal feature enhancement and fusion, Comput. Secur. 132 (2023) 103341.

[6] T. Peng, S. Chen, F. Zhu, J. Tang, J. Liu, X. Hu, PTLVD: Program slicing and transformer-based line-level vulnerability detection system, in: 2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation, SCAM, IEEE, 2023, pp. 162–173.

[7] M. Fu, C. Tantithamthavorn, Linevul: A transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories, 2022, pp. 608–620.

[8] D. Hin, A. Kan, H. Chen, M.A. Babar, LineVD: statement-level vulnerability detection using graph neural networks, in: Proceedings of the 19th International Conference on Mining Software Repositories, 2022, pp. 596–607.

[9] S. Khoshnood, M. Kusano, C. Wang, ConcBugAssist: constraint solving for diagnosis and repair of concurrency bugs, in: Proceedings of the 2015 International Symposium on Software Testing and Analysis, 2015, pp. 165–176.

[10] Github, Github, 2024, URL https://github.com. (Accessed 16 September 2024).

[11] N.V. Database, National vulnerability database, 2024, URL https://samate.nist.gov/SRD/index.php. (Accessed 16 September 2024).

[12] M. Schuster, K.K. Paliwal, Bidirectional recurrent neural networks, IEEE Trans. Signal Process. 45 (11) (1997) 2673–2681.

[13] FFmpeg, FFmpeg, 2024, URL https://ffmpeg.org. (Accessed 16 September 2024).

[14] OpenSSL, OpenSSL, 2024, URL https://www.openssl.org. (Accessed 16 September 2024).

[15] PostgreSQL, The world's most advanced open source relational database, 2024, URL https://www.postgresql.org. (Accessed 16 September 2024).

[16] A. Subversion, Apache subversion, 2024, URL https://subversion.apache.org. (Accessed 16 September 2024).

[17] W. Zheng, Y. Jiang, X. Su, VulSPG: Vulnerability detection based on slice property graph representation learning, 2021, CoRR abs/2109.02527 (2021), arXiv preprint arXiv:2109.02527.

[18] Flawfinder, Flawfinder, 2022, URL https://dwheeler.com/flawfinder. (Accessed 16 September 2024).

[19] Checkmarx, Checkmarx, 2022, URL https://www.checkmarx.com. (Accessed 16 September 2024).

[20] Y. Li, S. Wang, T.N. Nguyen, Vulnerability detection with fine-grained interpretations, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 292–303.

[21] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, H. Jin, Vuldeelocator: a deep learning-based fine-grained vulnerability detector, IEEE Trans. Dependable Secur. Comput. 19 (4) (2021) 2821–2837.

[22] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, Adv. Neural Inf. Process. Syst. 32 (2019).

[23] H. Wang, G. Ye, Z. Tang, S.H. Tan, S. Huang, D. Fang, Y. Feng, L. Bian, Z. Wang, Combining graph-based learning with automated data collection for code vulnerability detection, IEEE Trans. Inf. Forensics Secur. 16 (2020) 1943–1958.

[24] W. Wang, T.N. Nguyen, S. Wang, Y. Li, J. Zhang, A. Yadavally, DeepVD: Toward class-separation features for neural network vulnerability detection, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE, 2023, pp. 2249–2261.

[25] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, Vuldeepecker: A deep learning-based system for vulnerability detection, 2018, arXiv preprint arXiv:1801.01681.

[26] X. Li, B. Feng, G. Li, T. Li, M. He, A vulnerability detection system based on fusion of assembly code and source code, Secur. Commun. Netw. 2021 (1) (2021) 9997641.

[27] X. Cheng, G. Zhang, H. Wang, Y. Sui, Path-sensitive code embedding via contrastive learning for software vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 519–531.

[28] J. Tian, J. Zhang, F. Liu, Bbreglocator: A vulnerability detection system based on bounding box regression, in: 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops, DSN-W, IEEE, 2021, pp. 93–100.

[29] C. Pornprasit, C.K. Tantithamthavorn, Deeplinedp: Towards a deep learning approach for line-level defect prediction, IEEE Trans. Softw. Eng. 49 (1) (2022) 84–98.

[30] Y. Dong, Y. Tang, X. Cheng, Y. Yang, S. Wang, SedSVD: Statement-level software vulnerability detection based on relational graph convolutional network with subgraph embedding, Inf. Softw. Technol. 158 (2023) 107168.

[31] W. Bai, C. Du, L. Zhang, D. Wang, D. Liu, Fine-grained vulnerability location based on LLVM, ITAIC, in: 2023 IEEE 11th Joint International Information Technology and Artificial Intelligence Conference, vol. 11, IEEE, 2023, pp. 496–500.

[32] X. Lv, J. Fu, T. Peng, Vuloc: Vulnerability location framework based on assembly code slicing, 2024, Available at SSRN 4866850.

[33] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li, et al., DeepSeek-coder: When the large language model meets programming–the rise of code intelligence, 2024, arXiv preprint arXiv:2401.14196.

[34] S. Kang, J. Yoon, S. Yoo, Large language models are few-shot testers: Exploring llm-based general bug reproduction, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE, 2023, pp. 2312–2323.

[35] G. Lu, X. Ju, X. Chen, W. Pei, Z. Cai, GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning, J. Syst. Softw. 212 (2024) 112031.

[36] X. Du, G. Zheng, K. Wang, J. Feng, W. Deng, M. Liu, B. Chen, X. Peng, T. Ma, Y. Lou, Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG, 2024, arXiv preprint arXiv:2406.11147.

[37] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint arXiv:2002.08155.